



M

ARTIFICIAL INTELLIGENCE(Q) – MAY 2026
KECERDASAN TIRUAN(Q) – MEI 2026



Deep Sequence Model

Mohammad Iqbal, S.Si., M.Si., Ph. D.

OUTLINES



- ❑ Introduction
- ❑ *Sequence Modeling*
- ❑ Neurons with *recurrence*
- ❑ *Recurrence Neural Networks*
- ❑ Intuitive pf RNN
- ❑ Long Short Term Memory
- ❑ Fundamental *Attention*
- ❑ Intuitive of *Attention*





Introduction

Intuitively...

<https://www.pinterest.com/pin/666743919815539655/>



Given a taxi trip, **can you predict where it will go next?**

Turn right?



Go straight?



Turn Left?



Intuitively...

<https://www.pinterest.com/pin/666743919815539655/>



What do we need to predict the next destination?



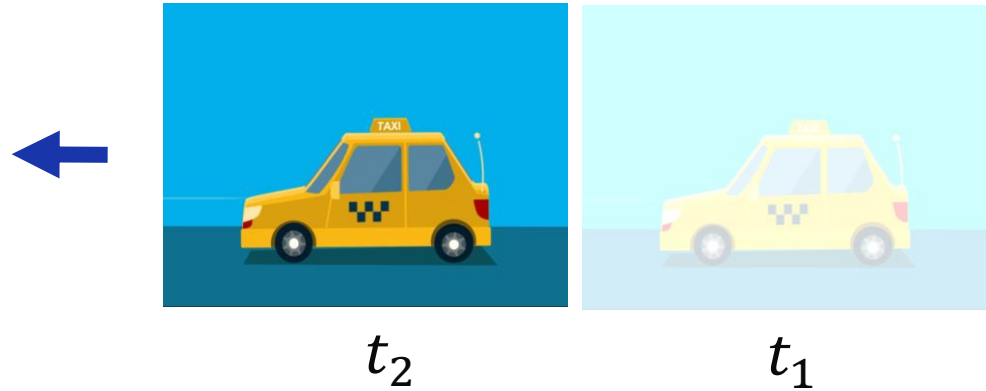
t_1

Intuitively...

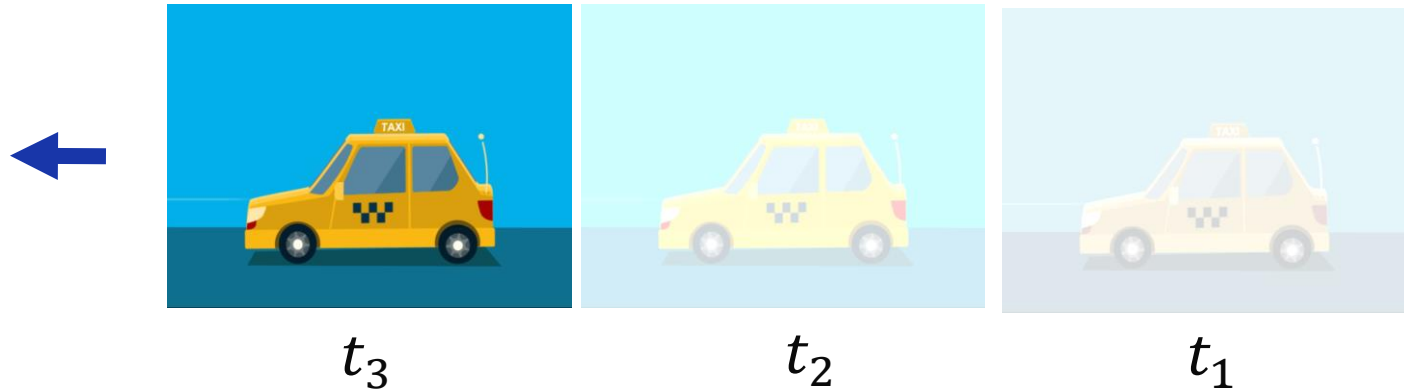
<https://www.pinterest.com/pin/666743919815539655/>



What do we need to predict the next destination?

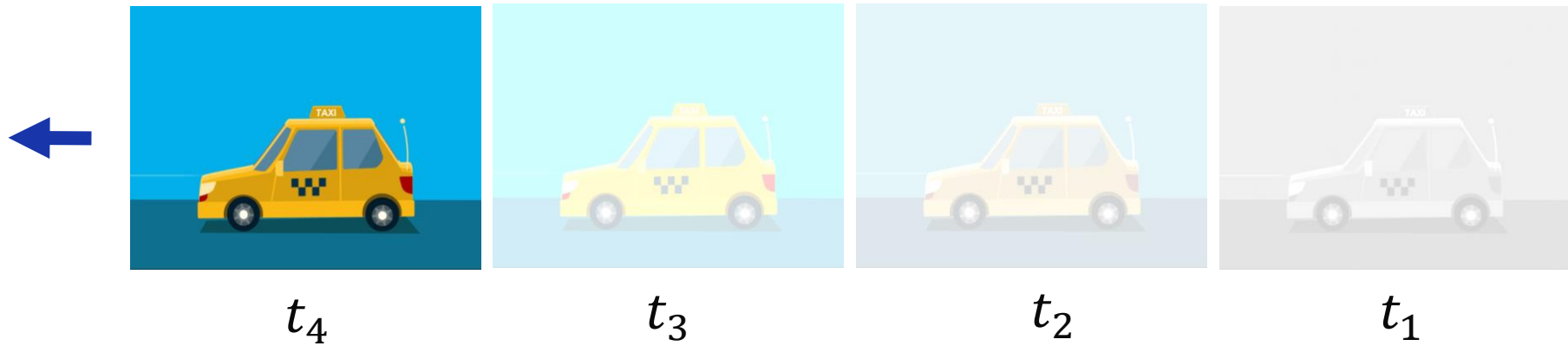


What do we need to predict the next destination?



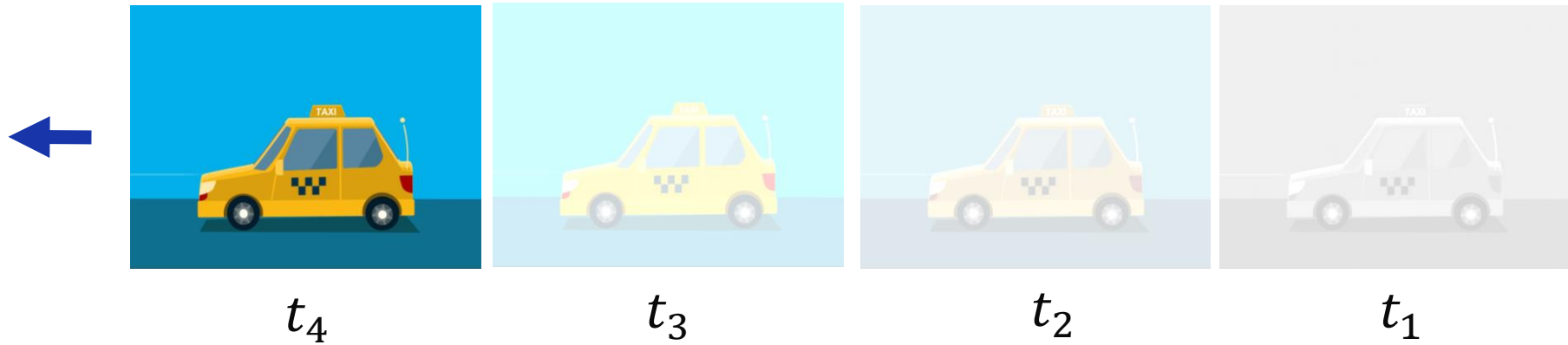
Historical taxi trips information

What do we need to predict the next destination?



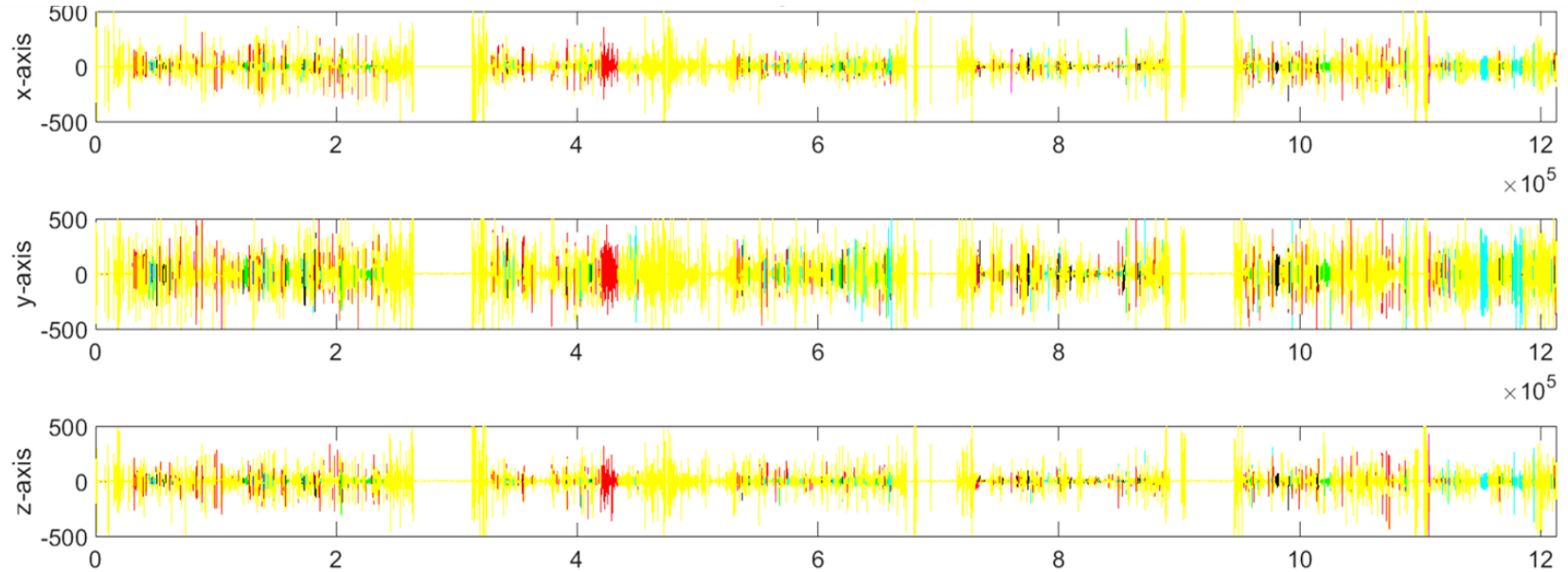
Historical taxi trips information

What do we need to predict the next destination?



Historical taxi trips information

Mostly sequence data in real life...



IMU Sensor

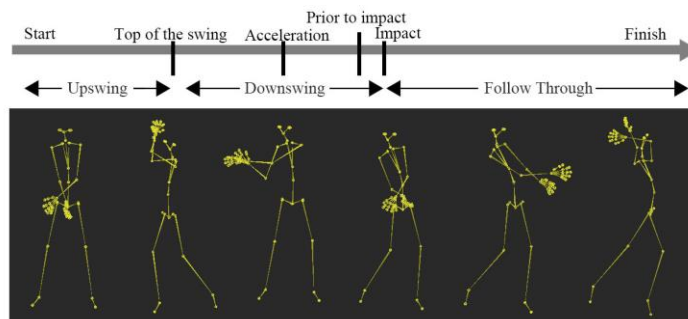
Sequence data...



DNA / RNA



Stock price

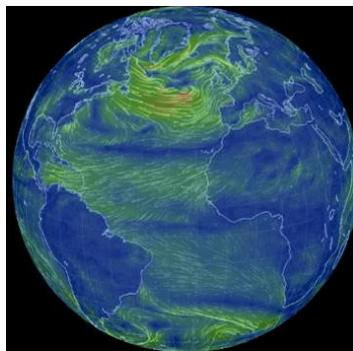


Video = Image sequence



Heart beat

Climate



<https://www.genengnews.com/topics/omics/dna/could-we-store-all-of-the-worlds-data-in-a-coffee-mug-full-of-dna/>
<https://qifer.com/en/A3iO>
<https://www.mdpi.com/1424-8220/20/22/6534>
<https://gfycat.com/gifs/search/heart+rate+monitor>
<https://qifer.com/en/6Nd6>

Sequence Modelling

Sequence Fashion in real life application

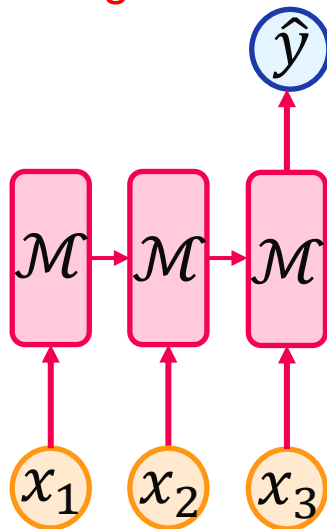


<http://introtodeeplearning.com/>

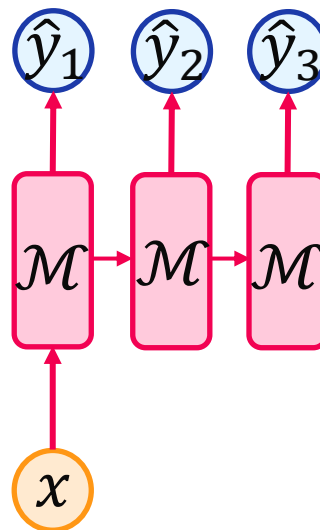
one-to-one



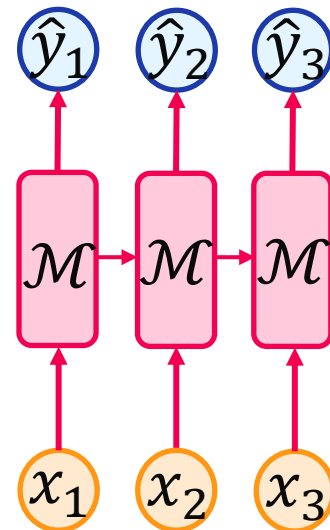
many-to-one



one-to-many



many-to-many



Nilai Siswa → Lulus?

Binary Classification

klo tujuan nya cari pengalaman sih boleh aja;
tapi belakangan banyak cerita unpaid internship yang merupakan
selubung modern slavery....



1



Sentiment Classification



"man in black shirt is playing guitar"



"construction worker in orange safety vest is working on road."



"two young girls are playing with lego toy."

Image Captioning



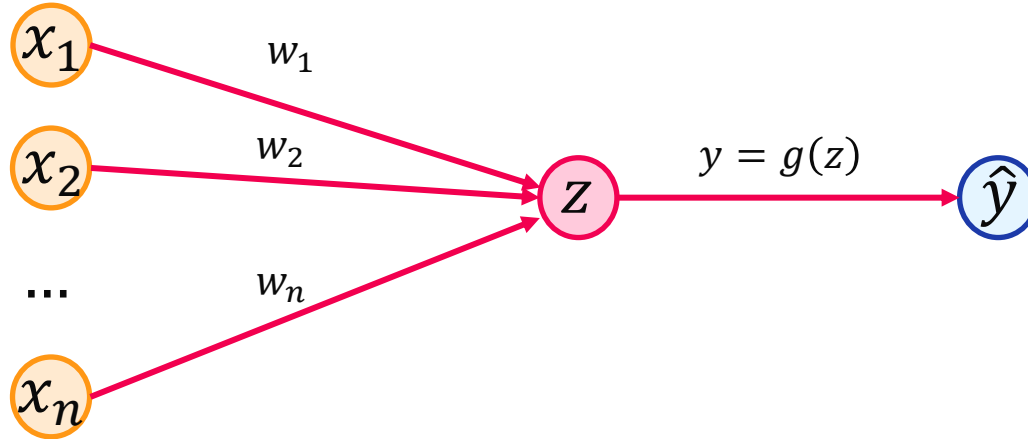
Machine Translation



Neurons with Recurrence

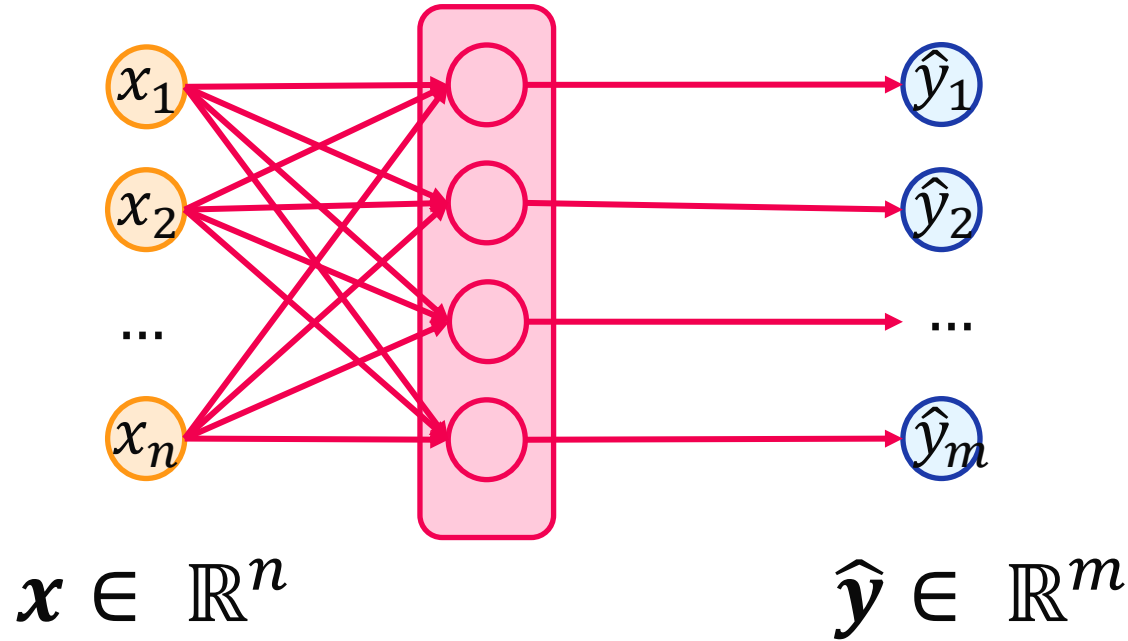
Perceptron (ingat)

<http://introtodeeplearning.com/>

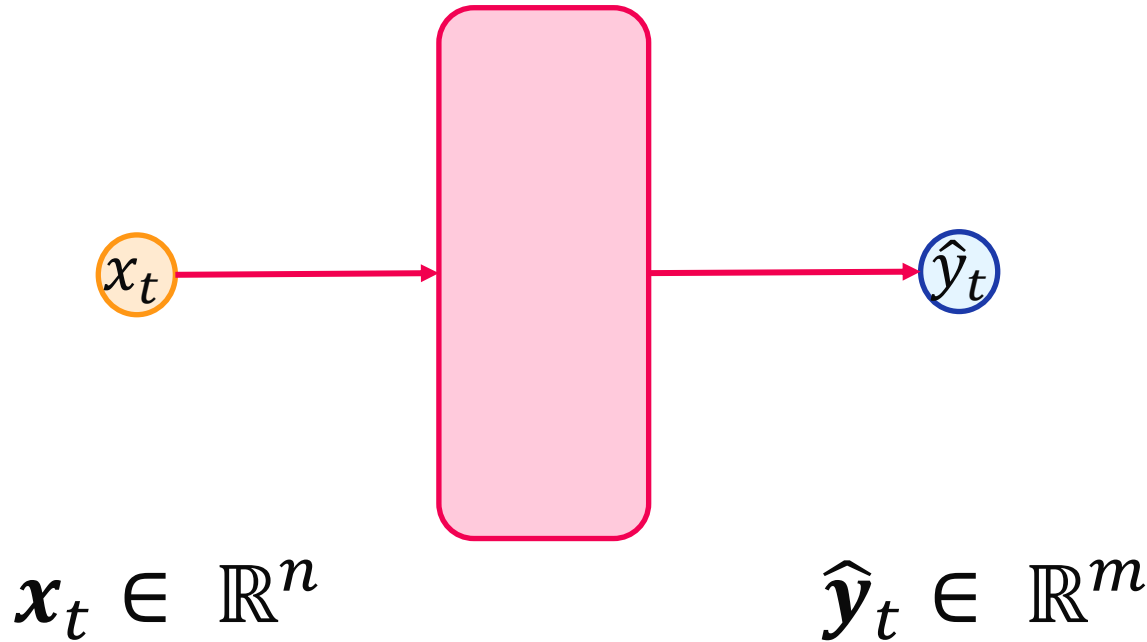


Feed Forward networks (ingat)

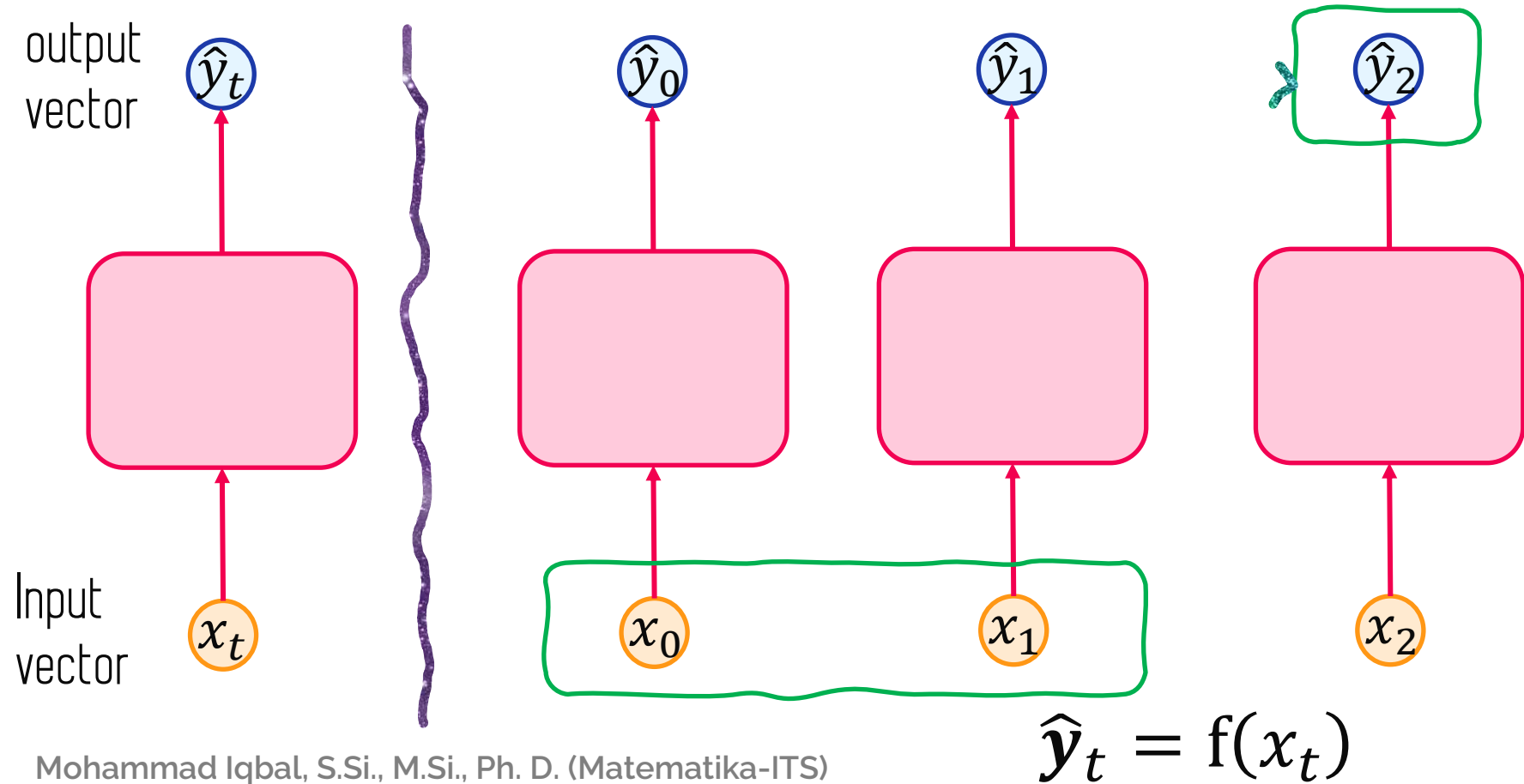
<http://introtodeeplearning.com/>



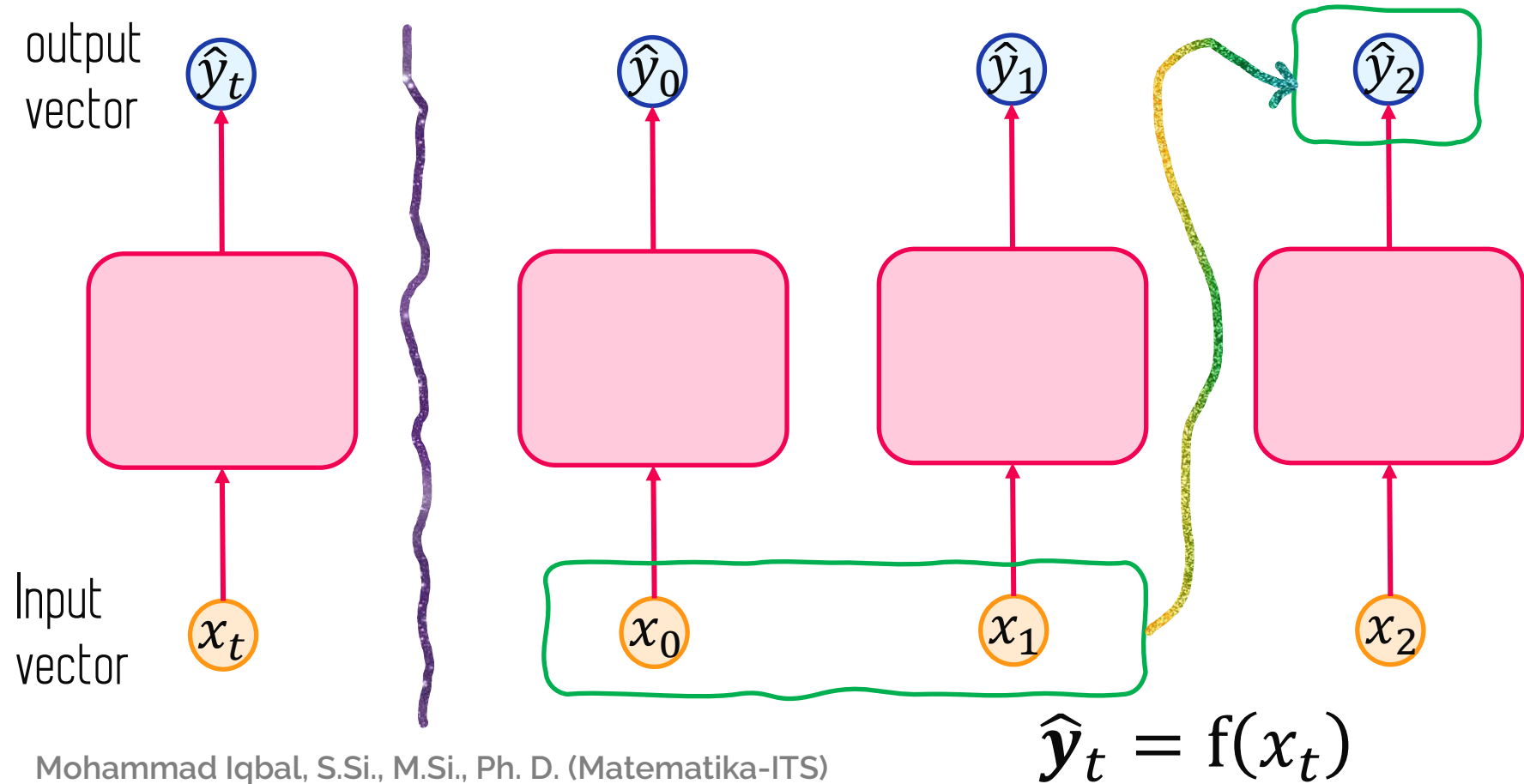
Feed Forward networks (recall)



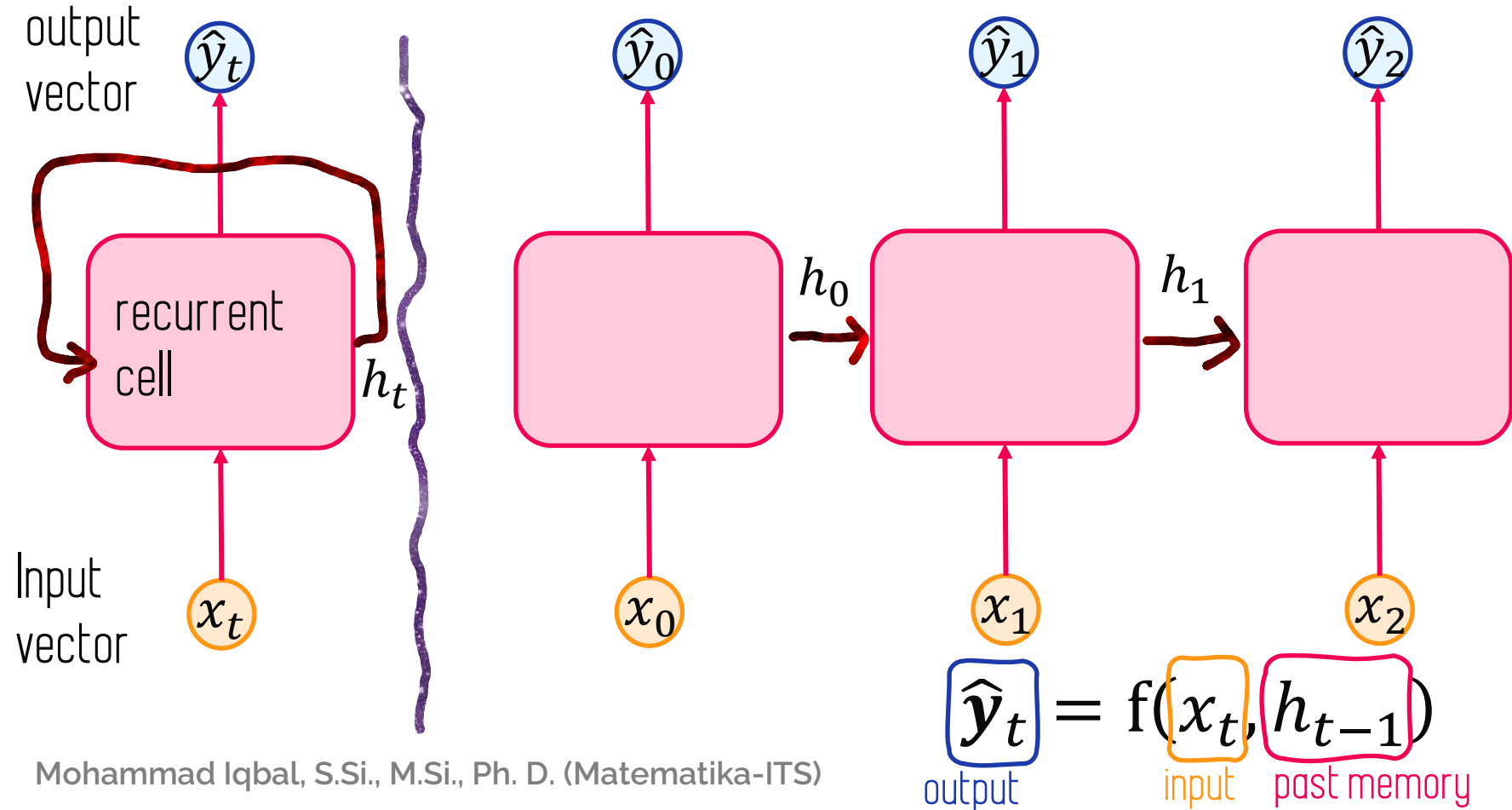
Individual time steps



Individual time steps



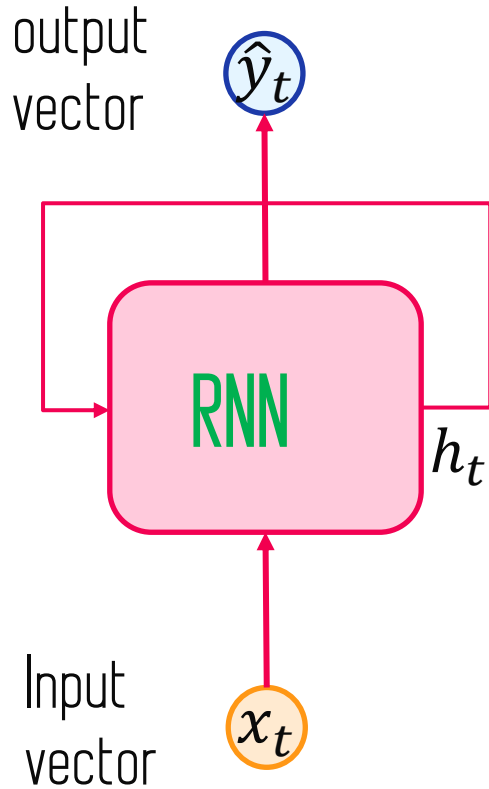
Neurons with recurrence





Recurrent Neural Networks

Recurrent neural networks (RNNs)



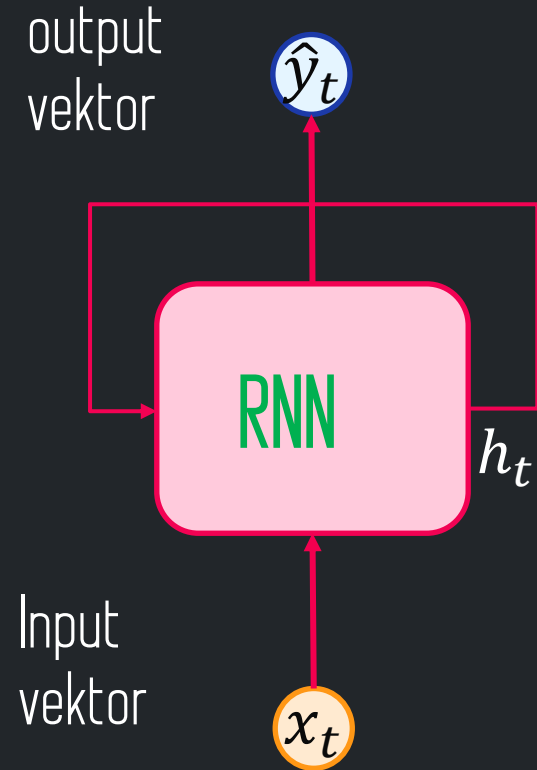
- Apply a **recurrence relation** for every time step to **process a sequence**.

$$\underset{\text{cell state}}{h_t} = \underset{\text{Function with weights } W}{f_W} \left(\underset{\text{input}}{x_t}, \underset{\text{Old state}}{h_{t-1}} \right)$$

- A **state** h_t that is updated in each time step
- Note: the same function and set of parameters are used at every time step

$$\hat{y}_t = f(x_t, h_{t-1})$$

Intuisi RNNs



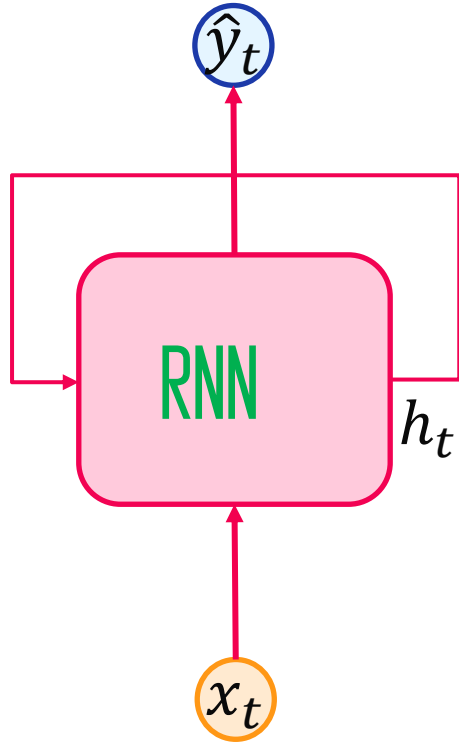
```
my_rnn = RNN()
hidden_state = [0, 0, 0, 0]

sentence = ["I", "love", "recurrent", "neural"]

for word in sentence:
    prediction, hidden_state = my_rnn(word, hidden_state)

next_word_prediction = prediction
# >>> "networks!"
```

Update and Output from RNN state



Output vector

$$\hat{y}_t = \mathbf{w}_{hy}^T h_t$$

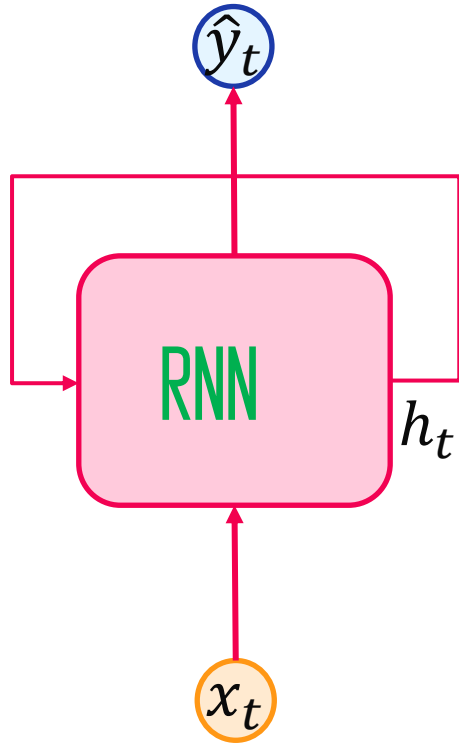
Update hidden state

$$h_t = \tanh(\mathbf{w}_{hh}^T h_{t-1} + \mathbf{w}_{xh}^T x_t)$$

Input vector

x_t

RNNs: Computational Graph Across Time

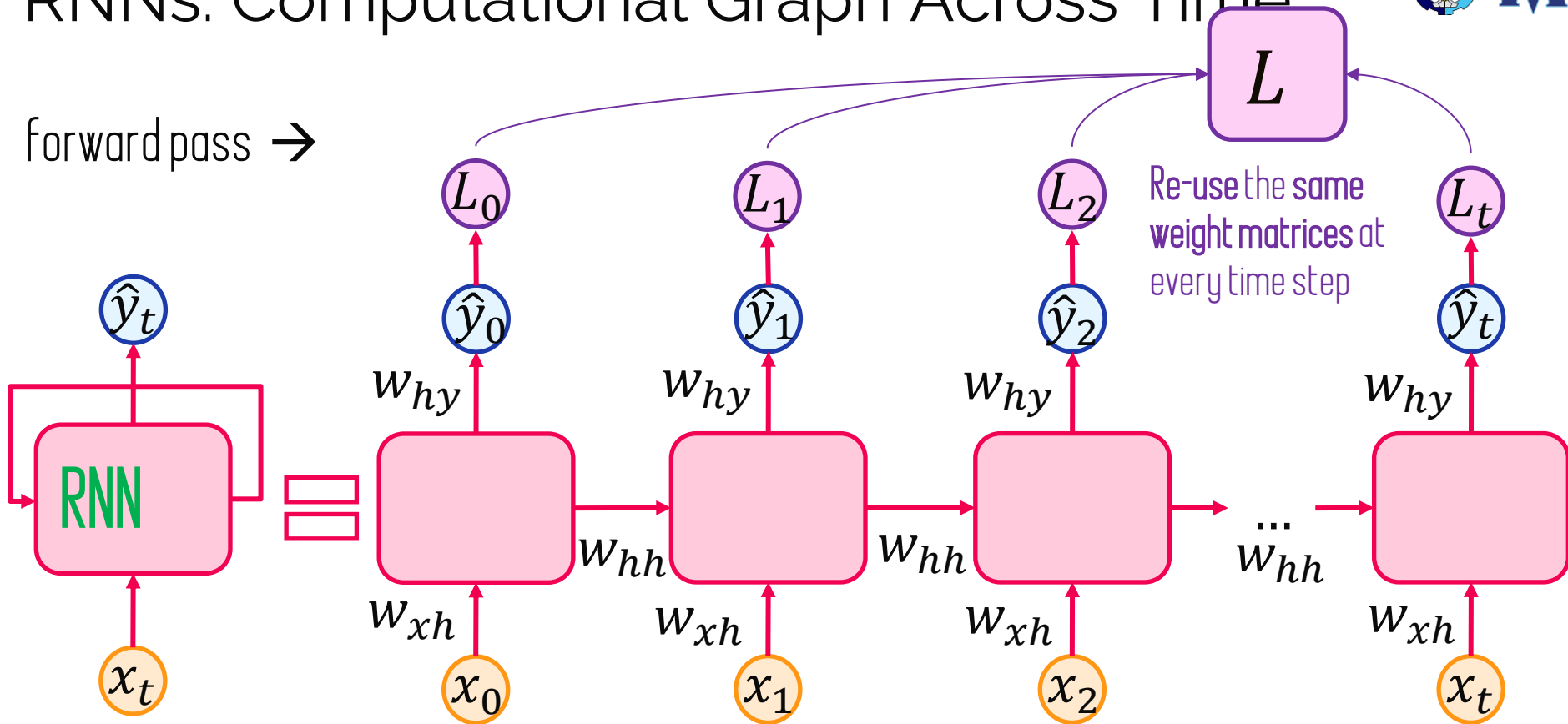


= represent as computational graph unrolled across time

RNNs: Computational Graph Across Time



forward pass $\rightarrow$



RNNs from scratch



```
import tensorflow as tf
import numpy as np

class cellRNNku(tf.keras.layers.Layer):
    def __init__(self, rnn_units, dim_input, dim_output):
        super(cellRNNku, self).__init__()

        # inisialisasi matriks bobot (parameter) rnn_units: Any
        self.W_xh = self.add_weight([rnn_units, dim_input])
        self.W_hh = self.add_weight([rnn_units, rnn_units])
        self.W_hy = self.add_weight([dim_output, rnn_units])

        # inisialisasi hidden state dengan nol
        self.h = tf.zeros([rnn_units, 1])

    def call(self, x):
        # update hidden state
        self.h = tf.math.tanh(self.W_hh * self.h + self.W_xh * x)

        # hitung output
        output = self.W_hy * self.h

        # return nilai output dan hidden state
        return output, self.h
```



$$h_t = \tanh(\mathbf{w}_{hh}^T h_{t-1} + \mathbf{w}_{xh}^T x_t)$$

$$\hat{y}_t = \mathbf{w}_{hy}^T h_t$$

RNN Implementation in Tensorflow



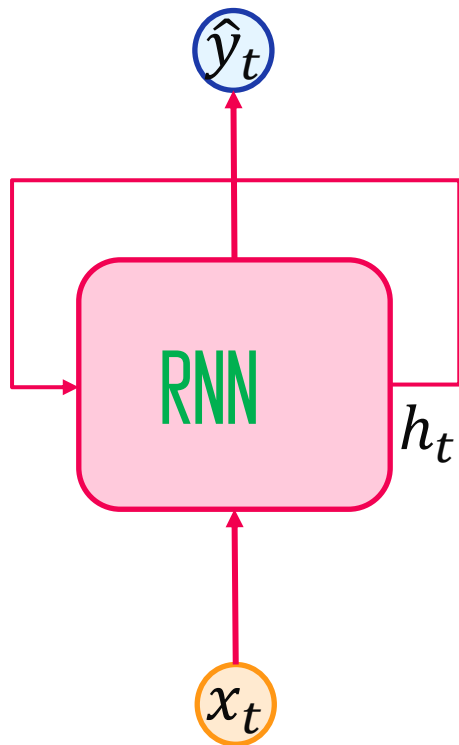
```
import tensorflow as tf
import numpy as np

inputs = np.random.random([32, 10, 8]).astype(np.float32)
simple_rnn = tf.keras.layers.SimpleRNN(4)

output = simple_rnn(inputs)
```



Sequence Modelling: Design Criteria



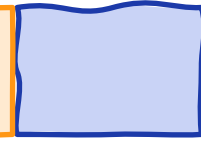
Four criteria to build a sequence model:

- Handle **variable-length sequence**
- Track **long-term dependencies**
- **Maintain information** about **order**
- **Share parameters** across the sequence

A sequence modeling problem: Predict next word



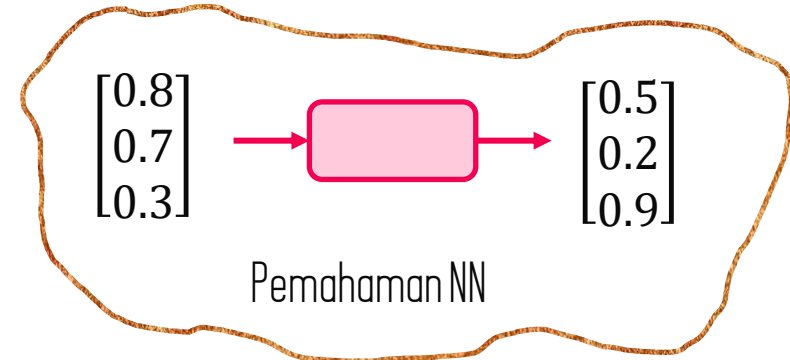
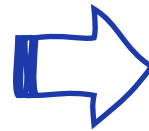
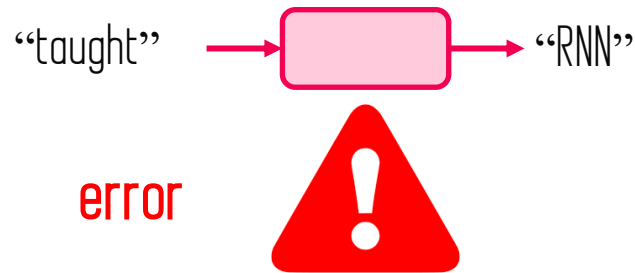
“This morning, I taught Artificial Intelligence Course for”



Given **these words** as an input

predict **the next word**

How to input text in neural networks?



Encode!

Encoding Language for a Neural Networks



Vocabulary: corpus of words

I this Intelligence
taught for morning
Artificial course RNN

Indexing: word to index

Artificial $\rightarrow 1$
Course $\rightarrow 2$
... $\rightarrow$...
this $\rightarrow N$

Embedding: index to fixed-sized vector

One-hot encoding

“Course” $\rightarrow [0, 1, 0, 0, 0]$

$\uparrow$
i-th index

Learned encoding

morning noon taught
supervise
I RNN
me CNN

Handle variable sequence lengths



“This morning, I taught Artificial Intelligence Course for RNN” → 10

“I ate with Rawon” → 4

“After class, assignment will be announced in classroom” → 9

Model Long-Term Dependencies



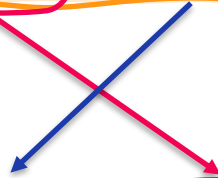
“Java is my homeland, but now I live on Taipei, Taiwan. I must fluently
speak _____”

- We need information from **the distant past** to predict the correct word precisely

Capture difference in sequence order



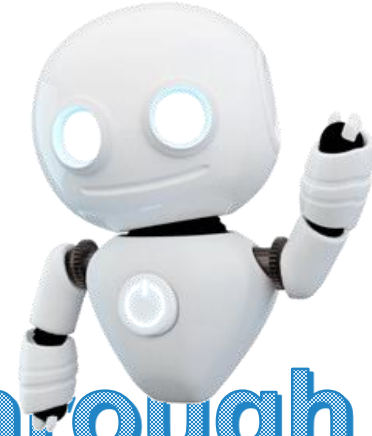
“The food was good, not bad at all”



“The food was bad, not good at all”

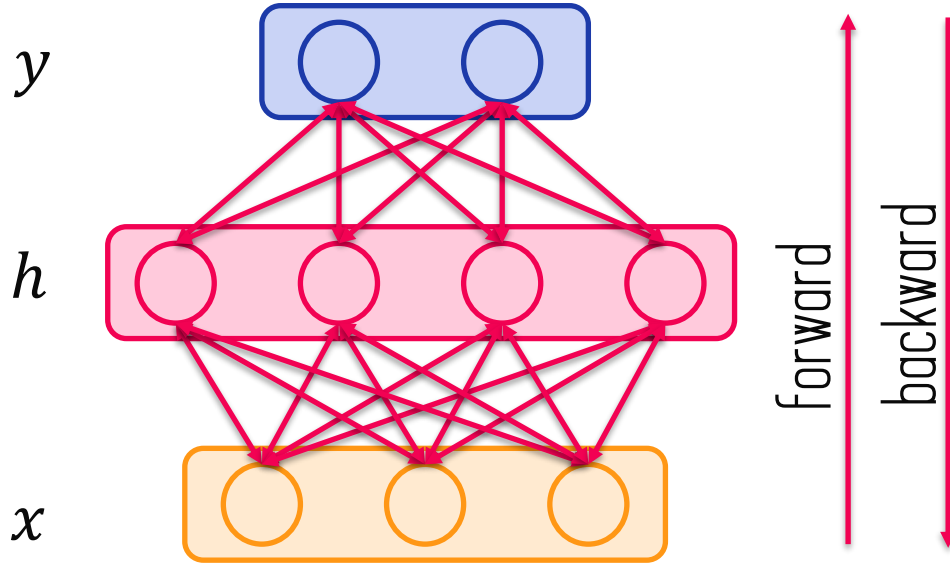


- Same words with **different position** → **different meaning!**



Backpropagation Through Time (BPTT)

Recall: Backpropagation in Feed Forward



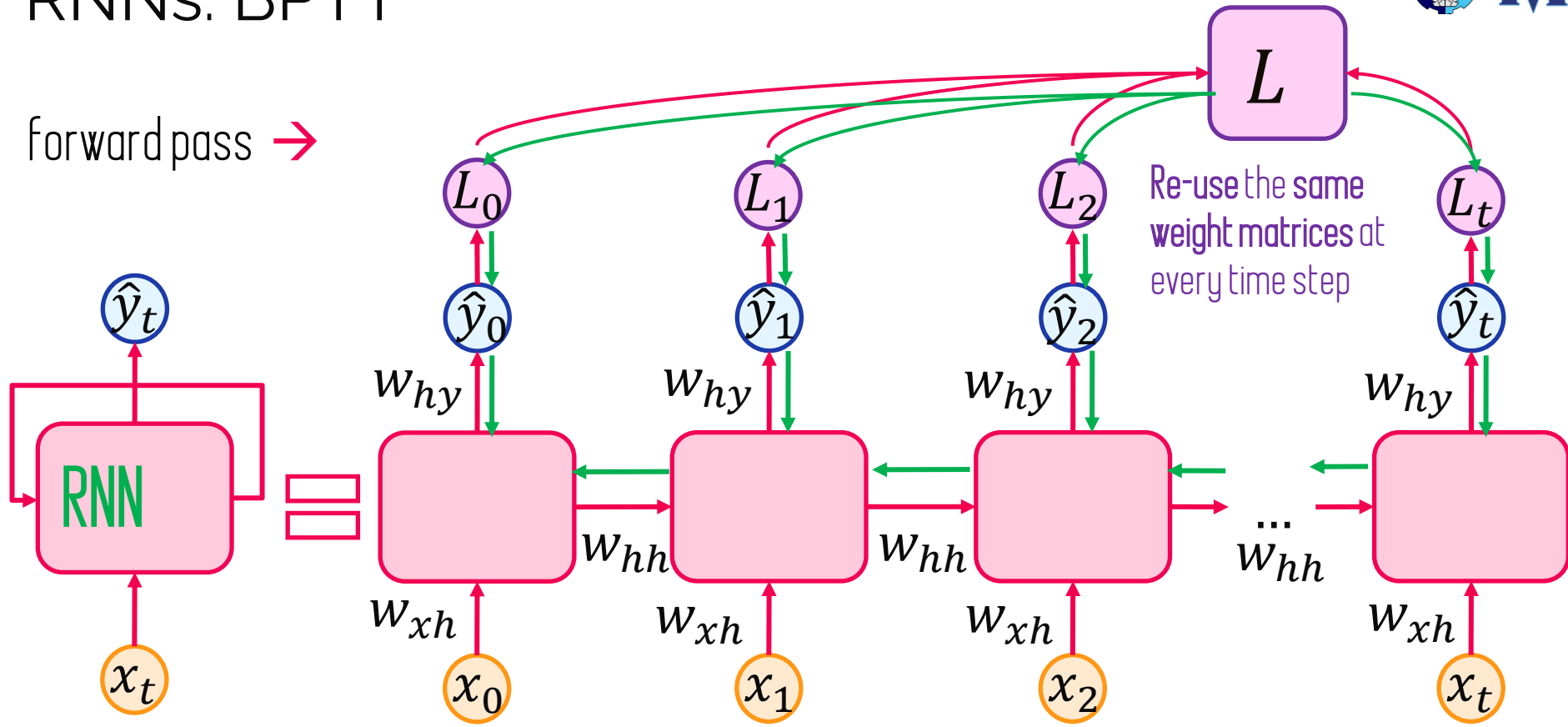
Backpropagation Algorithm:

1. Take the derivative (gradient) of the loss w.r.t. each parameter.
2. Shift parameters in order to minimize loss

RNNs: BPTT



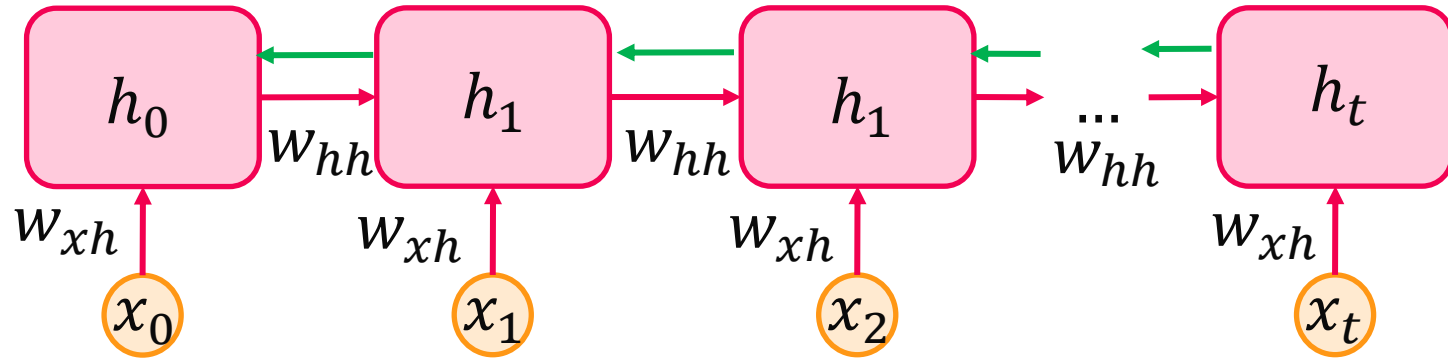
forward pass →



Re-use the same weight matrices at every time step

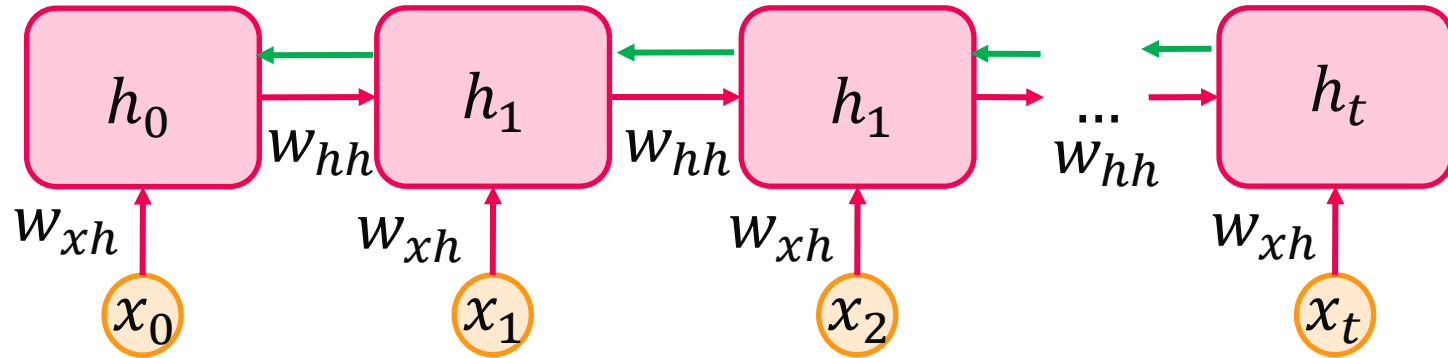
← backward pass

Standard RNN gradient flow



Computing the gradient h_0 involves many factors of w_{hh} + repeated gradient computation!

Standard RNN gradient flow

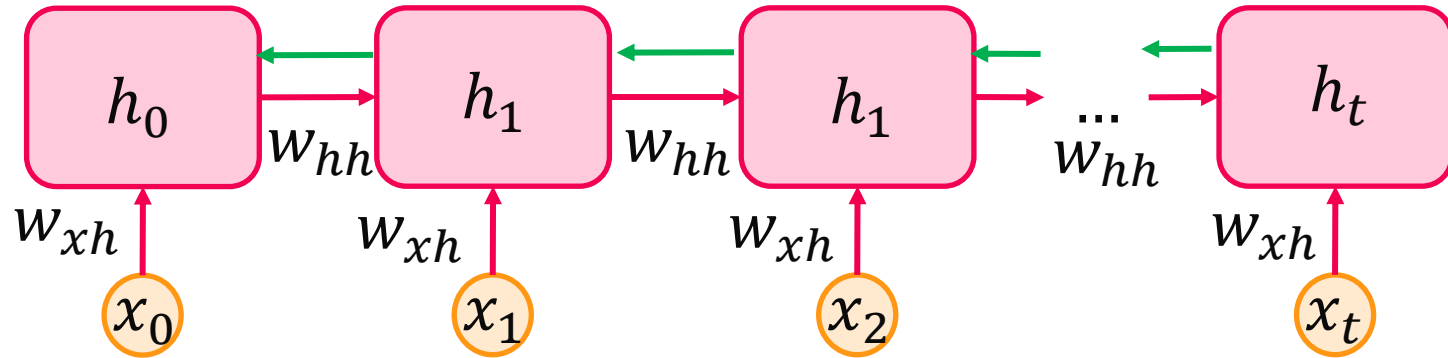


Computing the gradient h_0 involves many factors of w_{hh} + repeated gradient computation!

Many values > 1 : **exploding gradient!**

- **Gradient clipping** to scale big gradients

Standard RNN gradient flow



Computing the gradient h_0 involves many factors of w_{hh} + repeated gradient computation!

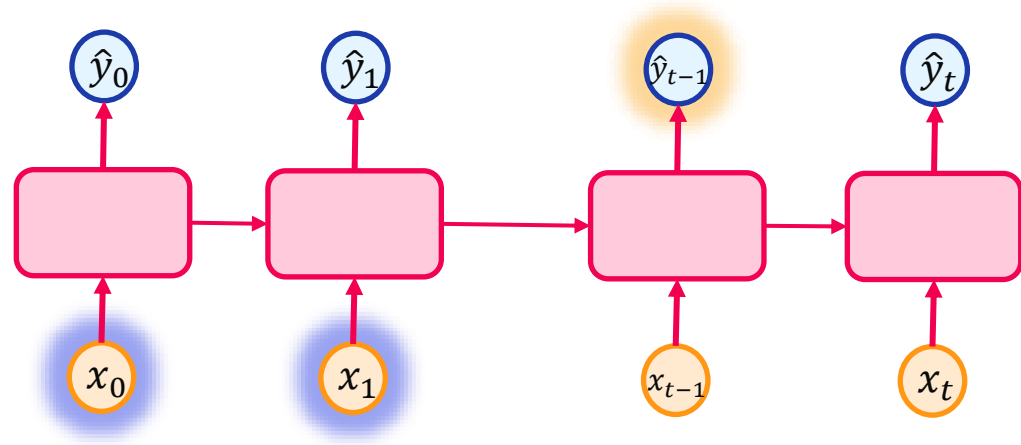
Many values < 1 : **vanishing gradient!**

1. Right **Activation function**
2. Right **Weight initialization**
3. Right **Network Architecture**

The problem of Long term dependencies

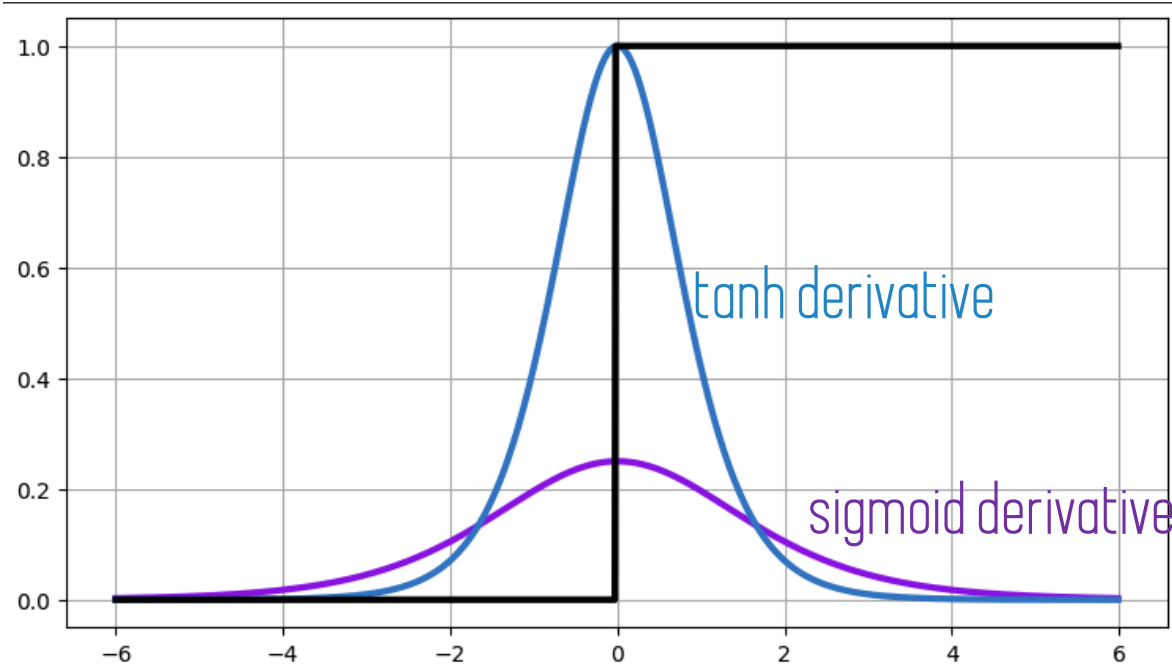
Why happens vanishing gradient:

- **Multiply many small numbers** together.
- Error due to further back time steps have **smallr and smaller gradients**
- **Bias parameters** to capture short-term dependencies.



Trick #1: Activation Function

<https://stats.stackexchange.com/questions/126238/what-are-the-advantages-of-relu-over-sigmoid-function-in-deep-neural-networks>



ReLU derivative

Use ReLU prevents f' dari
from shrinking the
gradients when $x > 0$

Trick #2: Parameter initialization

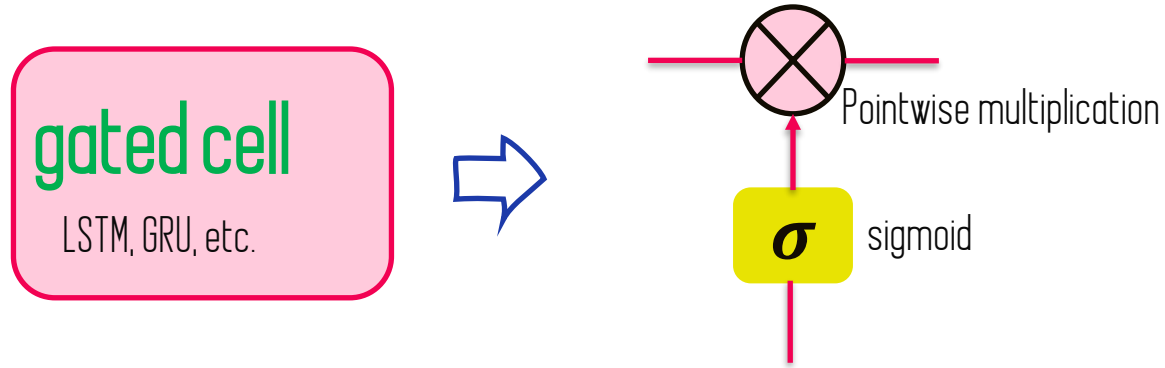
This helps to prevent the weight from shrinking to zero

- Initialize **weight** to **identity matrix**
- Initialize **biases** to **no1**

$$\begin{pmatrix} 1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 1 \end{pmatrix}$$

Trick #3: Gated cells

Idea : use **gates** to selectively **add** or **remove information** within each recurrent unit with.



LSTM rely on a gated cell to track information through many time step.

Gated LSTM cells **control information flow**:

1) Forget Gate 2) Store Gate 3) Update 4) Output



Long Short Term Memory (LSTM)

Key Concepts of LSTM:

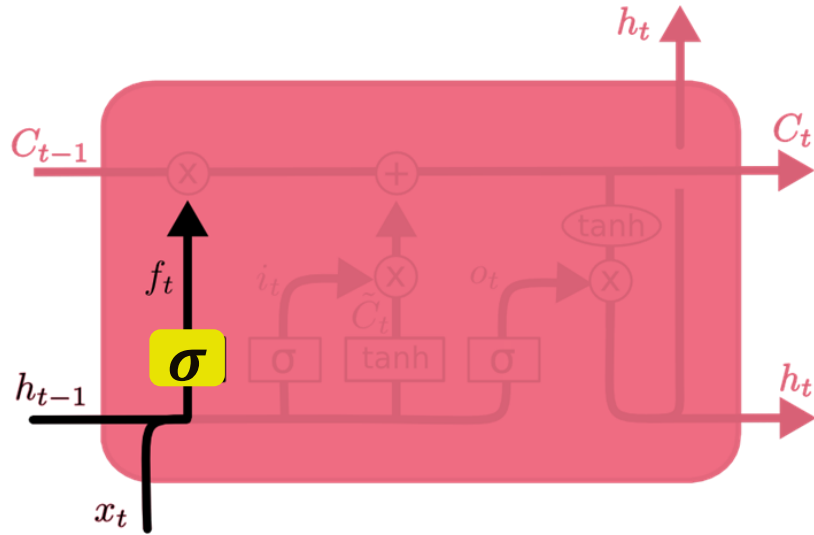
1. Maintain a **cell state**
2. Use **gates** to control the flow of information:
 - **Forget gate** can **delete irrelevant information**
 - **Store gate** save **relevant information from the input**
 - Selectively **update** cell state
 - **Output gate** returns a **filtered version of the cell state**
3. BPTT with partially **uninterrupted gradient flow**

Bedah LSTM

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>



Forget gate



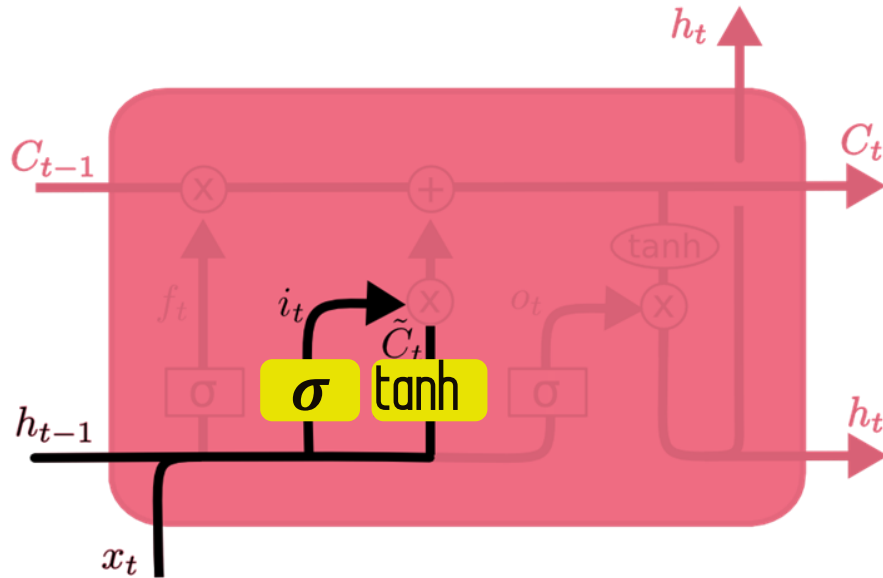
$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Bedah LSTM

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>



Input gate/Store gate



$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

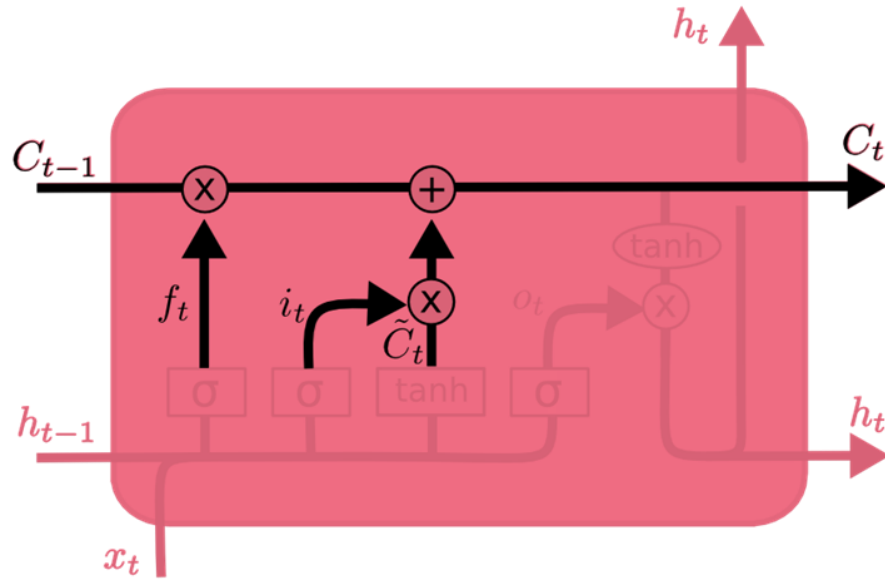
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Bedah LSTM

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>



Update cell state / memory cell state



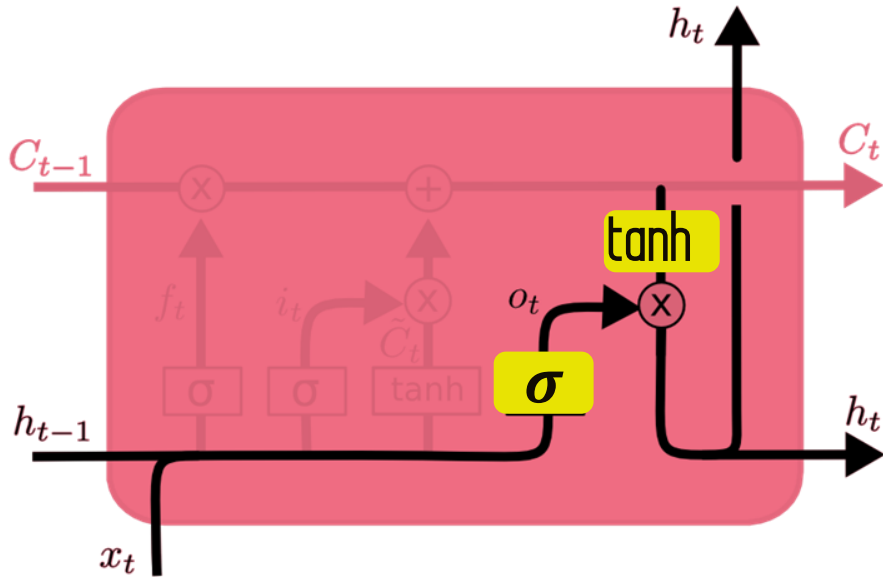
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Bedah LSTM

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>



Output gate



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

Tipe gated rnn

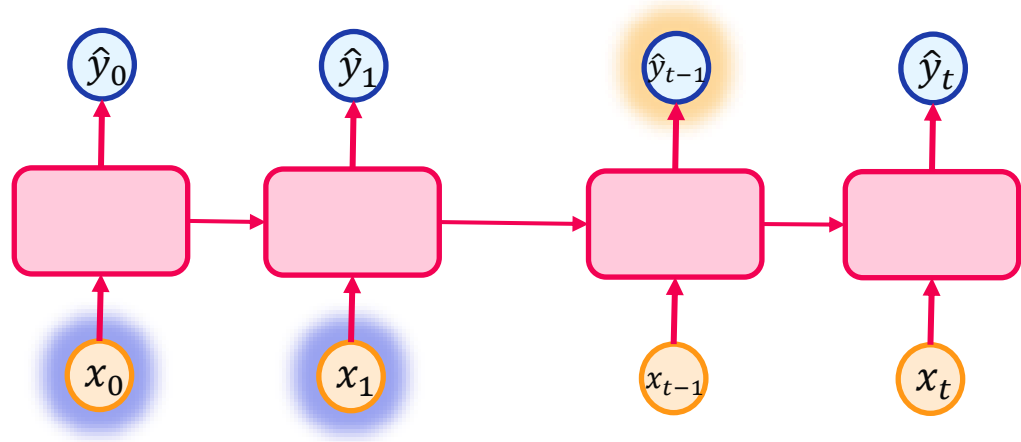


Bidirectional Maxout LSTM	Bidirectional Maxout GRU	Bidirectional Maxout DSGU	Bidirectional Maxout MGU	Bidirectional Maxout SMGU
Forget gate : f_j $= \sigma([W_f s]_j + [U_f h_{t-1}]_j)$ Input gate : $i_j = \sigma([W_i s]_j + [U_i h_{t-1}]_j)$ New Memory cell : $\tilde{c}_j = \max \begin{pmatrix} [W_c s]_j \\ + [U_c h_{t-1}]_j \end{pmatrix}$ Final memory cell : $c_j = f_j \odot c_{j-1} + i_j \odot \tilde{c}_j$ Output gate : o_j $= \sigma([W_o x]_j + [U_o h_{t-1}]_j)$ Final hidden state $h_j^{<t>} = o_j \odot \max(c_j)$ F. A. Gers, J. Schmidhuber. (2000). "Recurrent Nets That Time and Count". IJCNN.	Reset gate : $r_j = \sigma([W_r s]_j + [U_r h_{t-1}]_j)$ Update gate : $z_j = \sigma([W_z s]_j + [U_z h_{t-1}]_j)$ Candidate hidden layer : $\tilde{h}_t$ $= \max \begin{pmatrix} [W s]_j \\ + [U(r \odot h_{t-1})]_j \end{pmatrix}$ Final memory : $h_j^{<t>} = z_j \odot h_j^{<t-1>} + (1 - z_j) \odot \tilde{h}_j^{<t>}$ K. Cho et al. (2014). "Learning Phrase Representation using RNN Encoder-Decoder for Statistical Machine Translation". EMNLP.	Input : $s_g = W_h s + b_g$ Update gate : $z_{gj} = \sigma([W_{zg}]_j s_g \odot h_{t-1})$ z_{outj} $= \sigma([W_{out}]_j \max(z_g \odot h_{t-1}))$ z_j $= \sigma([W_z s]_j + [W h_{t-1}]_j + b_z)$ Final hidden state : $h_j^{<t>} = z_j \odot z_{out}$ $+ (1 - z_j) \odot h_j^{<t-1>}$ Y. Gao et al. (2016). "Deep Gate Recurrent Neural Network". JMLR	Update gate : z_j $= \sigma([W_z s]_j + [U_z h_{t-1}]_j + b_z)$ Candidate hidden layer : $\tilde{h}_t$ $= \max \begin{pmatrix} [W s]_j \\ + [U(z \odot h_{t-1})]_j \\ + b \end{pmatrix}$ Final memory : $h_j^{<t>} = z_j \odot h_j^{<t-1>} + (1 - z_j) \odot \tilde{h}_j^{<t>}$ G. B. Zhou et al. (2011). "Minimal Gated Unit for Recurrent Neural Networks". ICML	Update gate : $z_j = \sigma([U_z h_{t-1}]_j)$ Candidate hidden layer : $\tilde{h}_t$ $= \max \begin{pmatrix} [W s]_j \\ + [U(z \odot h_{t-1})]_j \\ + b \end{pmatrix}$ Final memory : $h_j^{<t>} = z_j \odot h_j^{<t-1>} + (1 - z_j) \odot \tilde{h}_j^{<t>}$ J. Heck, F. M. Salem. (2017). "Simplified Minimal Gated Unit Variations for Recurrent Neural Networks". IEEE 60th International Midwest Symposium on Circuits and Systems
Mohammad Iqbal, S	Si., M.Si., Ph. D. (Matematika-ITS)			

Goal of Sequence model

Limitations of RNN

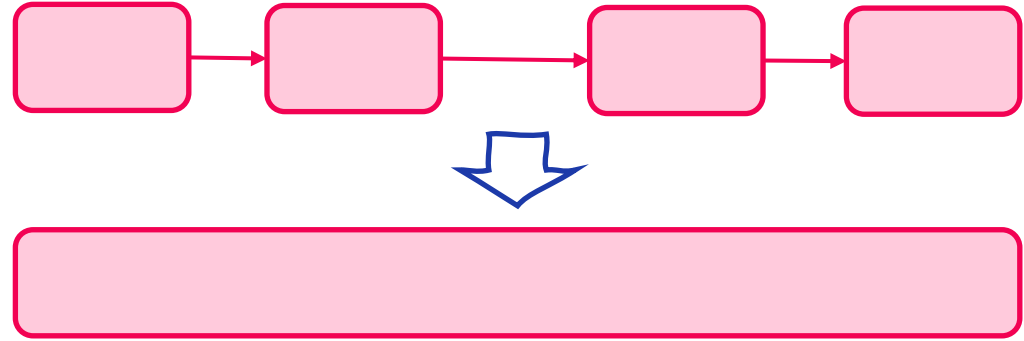
1. Encoding bottleneck
2. Slow, no parallelization
3. Not long memory



Goal of Sequence model

Desired Capabilities

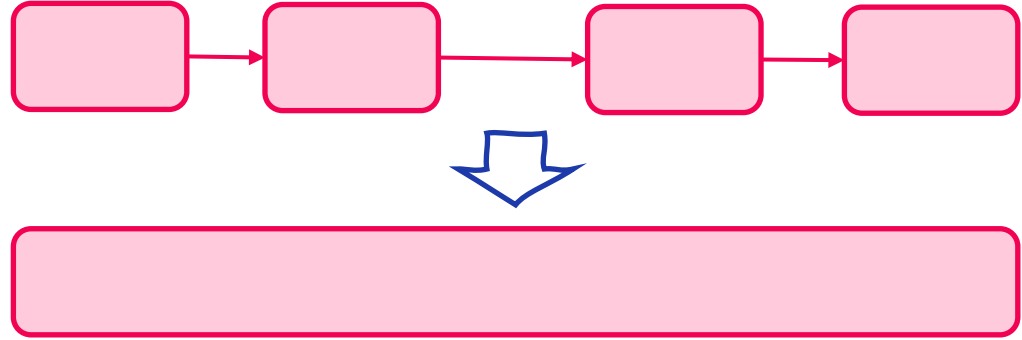
1. Continuous stream
2. Parallelization
3. Long memory



Goal of Sequence model

Can we eliminate the need for recurrence entirely?

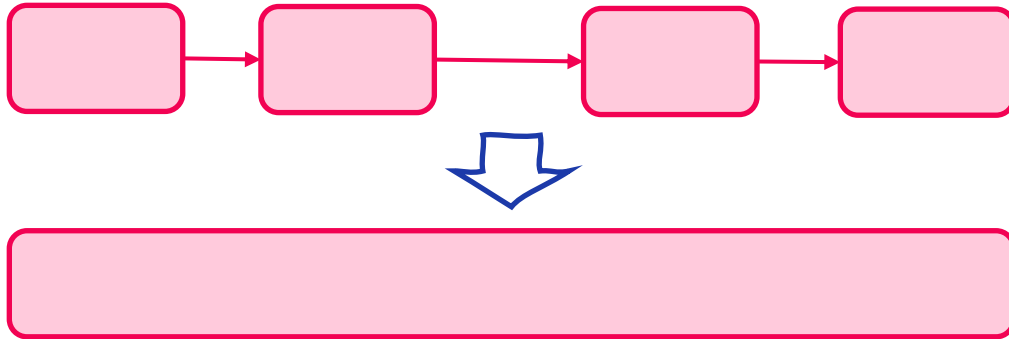
Common Idea: Feed everything into dense networks



1. No recurrence
2. Not scalable
3. No order
4. No Long memory

Briliant yet Simple Idea: **Identify** and **attend to what is important!**

Linear Algebra is the tool!



A list of block matrices



A specific matrix



Attention is all you need!

Intuitively, Self-Attention

<http://introtodeeplearning.com/>



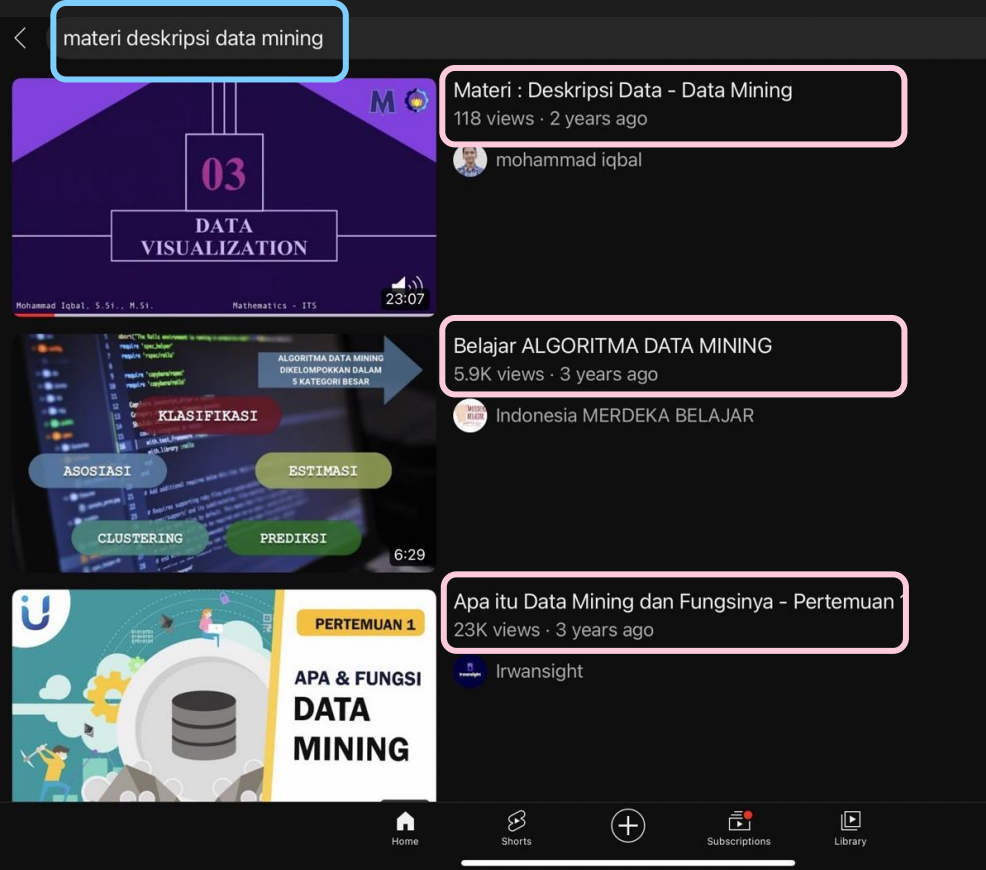
Attending the most important parts of an input



Search problem, right?

1. Identify which part to attend to
2. Extract the features with high attention

Example of Searching



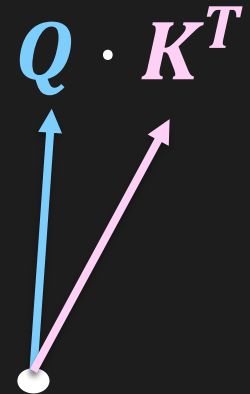
Query $[Q]$

Key $[K_1]$

Key $[K_2]$

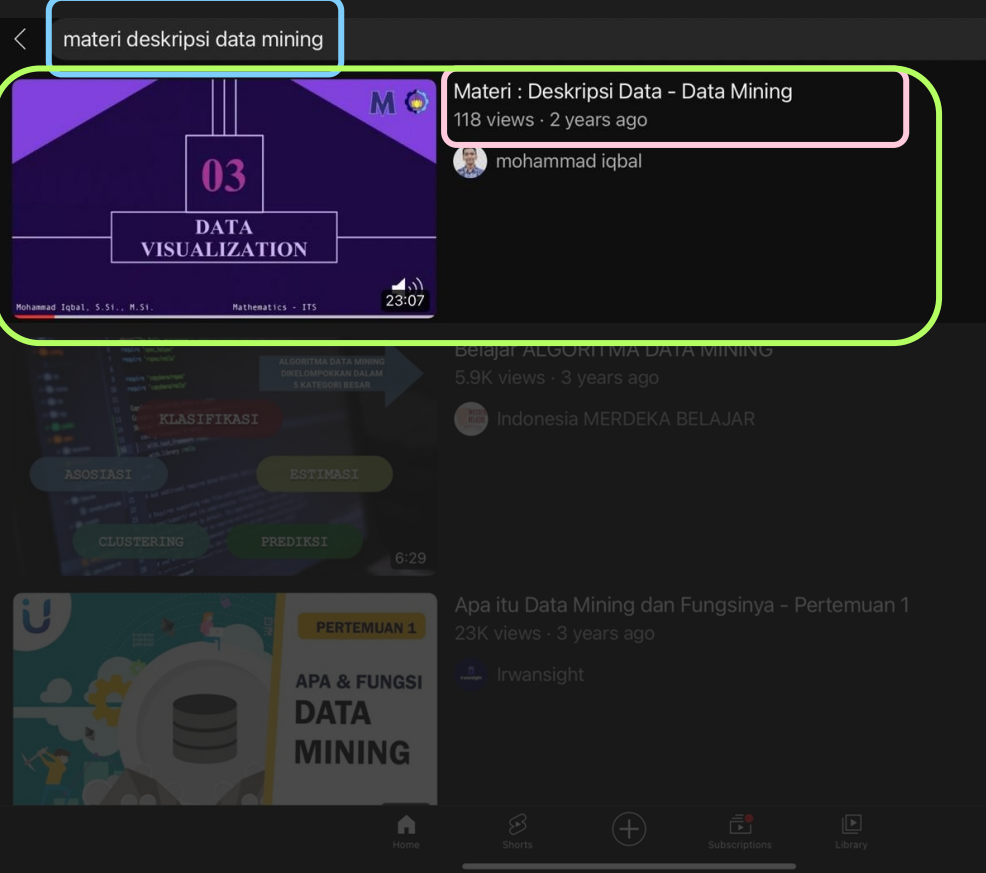
Key $[K_3]$

Cosine similarity

$$Q \cdot K^T$$


Compute attention mask: How similar is each key to the desired query?

Example of Searching



Query (Q)

Key (K_1)

Value (V)

Extract values based on attention: return the values of highest attention

Learning self-attention with NNs

<http://introtodeeplearning.com/>

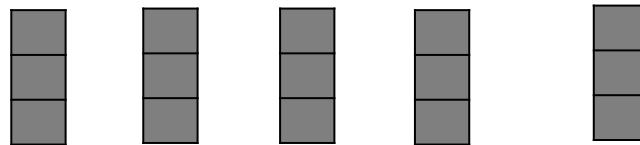


Goal: identify and attend to most important features in input

1. Encode **position** information

x

saya lupa bawa bekal sarapan

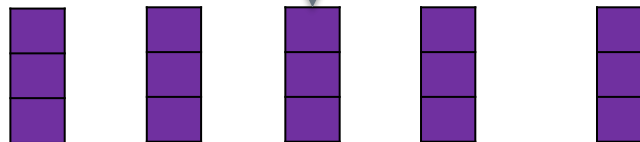


embedding



p_0 p_1 p_2 p_3 p_4

Position
information



Position
embedding

Position-aware encoding

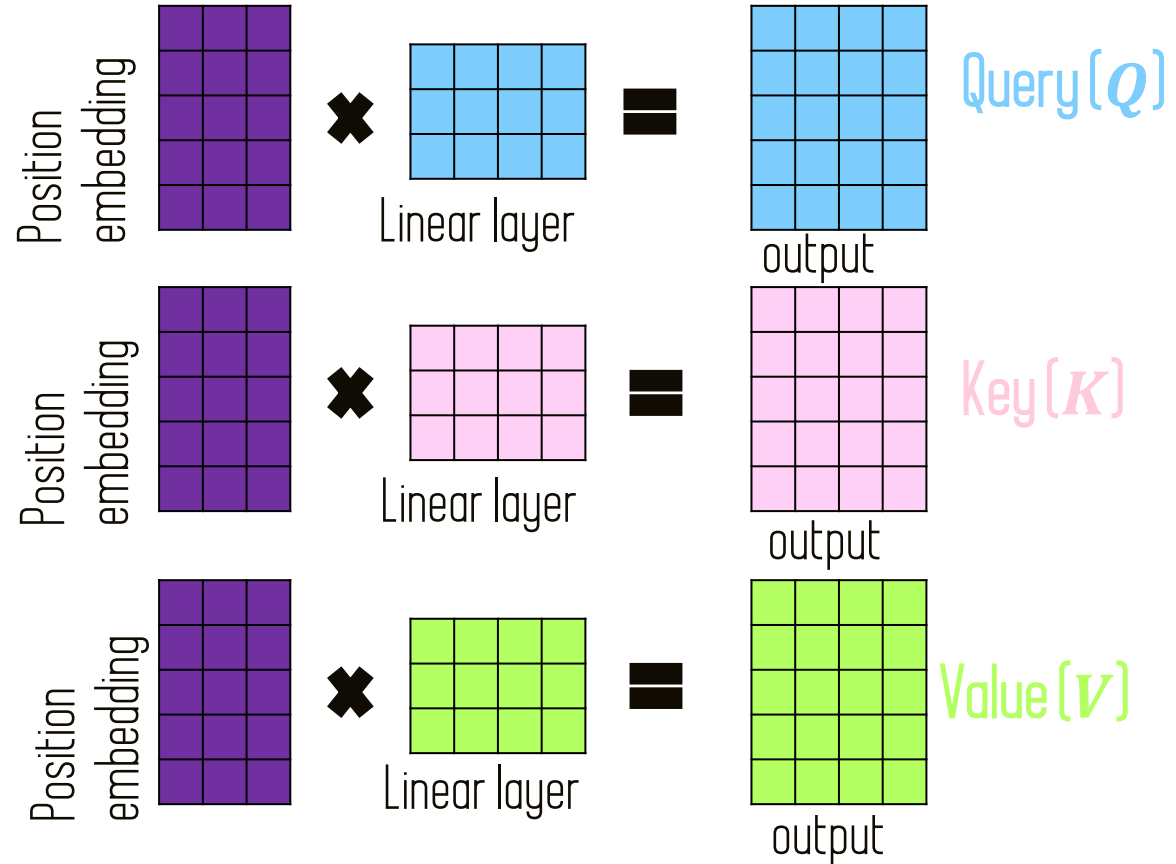
- Data is fed in all at once!
- Need to encode position information to understand order

Learning self-attention with NNs

<http://introtodeeplearning.com/>



1. Encode **position** information
2. Extract **query, key, value**, for search



Learning self-attention with NNs

<http://introtodeeplearning.com/>



1. Encode **position** information
2. Extract **query, key, value**, for search
3. Compute **attention weighting**

$$\text{softmax} \left(\frac{Q \cdot K^T}{\text{scaling}} \right)$$

Bobot Attention

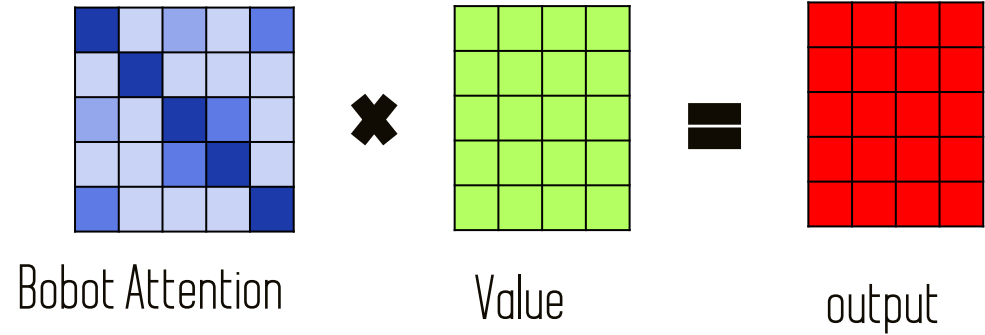
	saya	lupa	bawa	bekal	sarapan
saya					
lupa					
bawa					
bekal					
sarapan					

Learning self-attention with NNs



<http://introtodeeplearning.com/>

1. Encode **position** information
2. Extract **query, key, value**, for search
3. Compute **attention weighting**
4. Extract **features with high attention**



$$\text{softmax}\left(\frac{Q \cdot K^T}{\text{scalling}}\right) \cdot V = \mathbf{A}(Q, K, V)$$

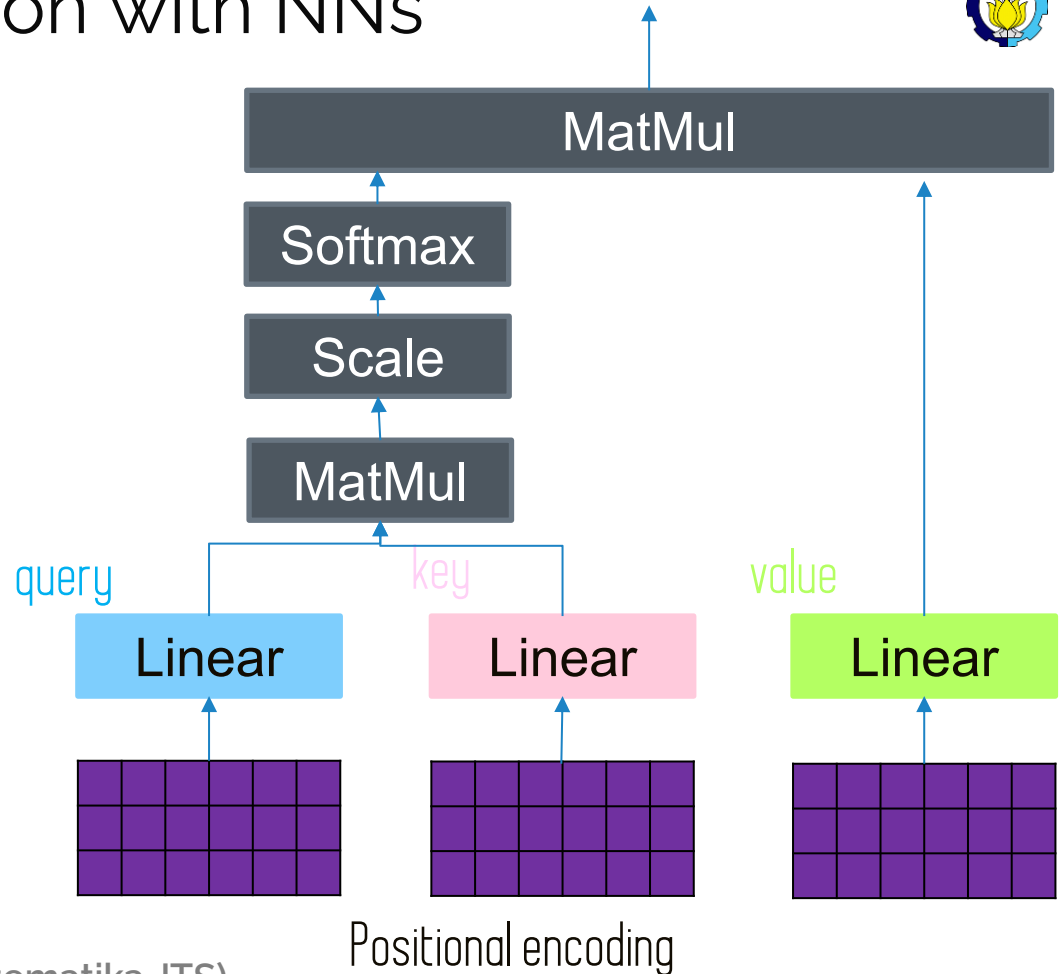
Learning self-attention with NNs

<http://introtodeeplearning.com/>



$$\text{softmax}\left(\frac{Q \cdot K^T}{\text{scaling}}\right) \cdot V$$

- These operations form a self-attention head that can plug into a larger network.
- Each head attends to a different part of input.



Multi Self-Attention Heads

<http://introtodeeplearning.com/>



Attention weighting

×



Value

=



Output



Output of attention head 1



Output of attention head 2



Output of attention head 3

Discussion...



Q n A